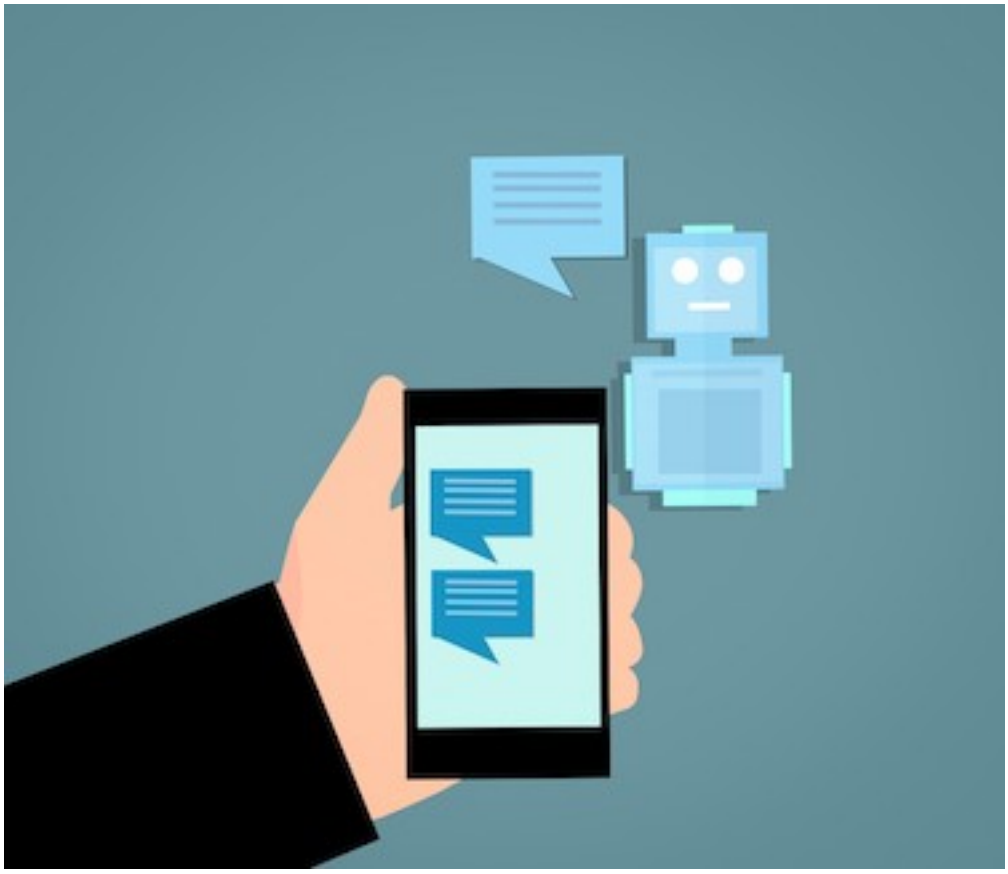
Create Simple Chatbot with Open Source LLMs using Python and Hugging Face

In this lab, you will create a very simple but functional chatbot!



Learning outcomes:

At the end of this lab, you will be able to:

- Describe the main components of a chatbot
- Explain what an LLM is
- Select an LLM for your application

- Feed input into a transformer (tokenization)
- Program your own simple chatbot in Python

Introduction: Under the hood of a chatbot

Intro: How does a chatbot work?

A chatbot is a computer program that takes a text input, and returns a corresponding text output.

Chatbots use a special kind of computer program called a transformer, which is like its brain. Inside this system is a language model (LLM), which is the core component that generates resposes.This helps the chatbot understand and generate human-like responses. It deciphers many examples of human conversations it has seen prior to responding in a sensible manner.

Transformers and LLMs work together within a chatbot to enable conversation. Here's a simplified explanation of how they interact:

- **Input processing:** When you send a message to the chatbot, the transformer helps process your input. It breaks down your message into smaller parts and represents them in a way that the chatbot can understand. Each part is called a token.

- **Understanding context:** The transformer passes these tokens to the LLM, which is a language model trained on lots of text data. The LLM has learned patterns and meanings from this data, so it tries to understand the context of your message based on what it has learned.

- **Generating response:** Once the LLM understands your message, it generates a response based on its understanding. The transformer then takes this response and converts it into a format that can be easily sent back to you.

- **Iterative conversation:** As the conversation continues, this process repeats. The transformer and LLM work together to process each new input message, understand the context, and generate a relevant response.

The key is that the LLM learns from a large amount of text data to understand language patterns and generate meaningful responses. The transformer helps with the technical aspects of processing and representing the input/output data,

Once the chatbot understands your message, it uses the language model to generate a response that it thinks will be helpful or interesting to you. The response is sent back to you, and the process continues as you have a back-and-forth conversation with the chatbot.

Intro: Hugging Face

Hugging Face is an organization that focuses on natural language processing (NLP) and AI. They provide a variety of tools, resources, and services to support NLP tasks.

You'll be making use of their Python library `transformers` in this project.

Alright! Now that you know how a chatbot works at a high level, let's get started with implementing a simple chatbot!

Step 1: Installing requirements

Follow these steps to create a Python virtual environment and install the necessary libraries. Open a new terminal first. Set up your virtual environment:

bash

```
pip3 install virtualenv
virtualenv my_env # create a virtual environment my_env
source my_env/bin/activate # activate my_env
```

Run

For this example, we use the transformers library, an open-source NLP toolkit, along with PyTorch (torch) for deep learning, while accelerate helps run AI models efficiently on CPU/GPU and numpy supports fast numerical and array computations in Python.

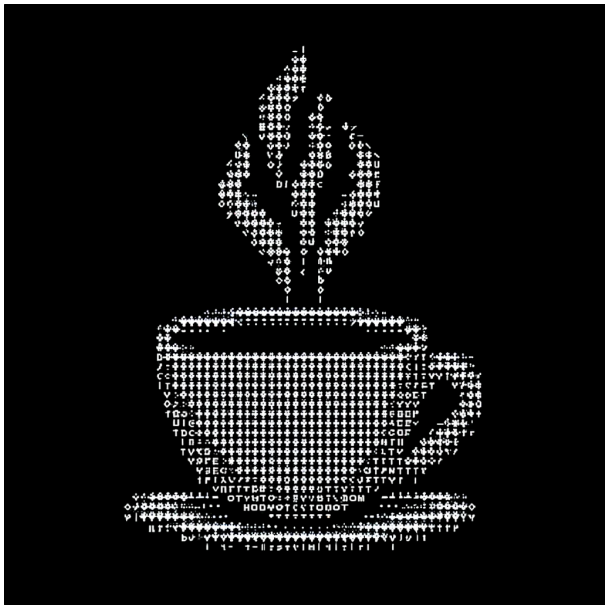
We pin library versions to ensure the code runs consistently without breaking due to future updates or changes in dependencies.

bash

```
pip install transformers==4.41.2 torch==2.2.2 accelerate==0.30.1 numpy==1.
```

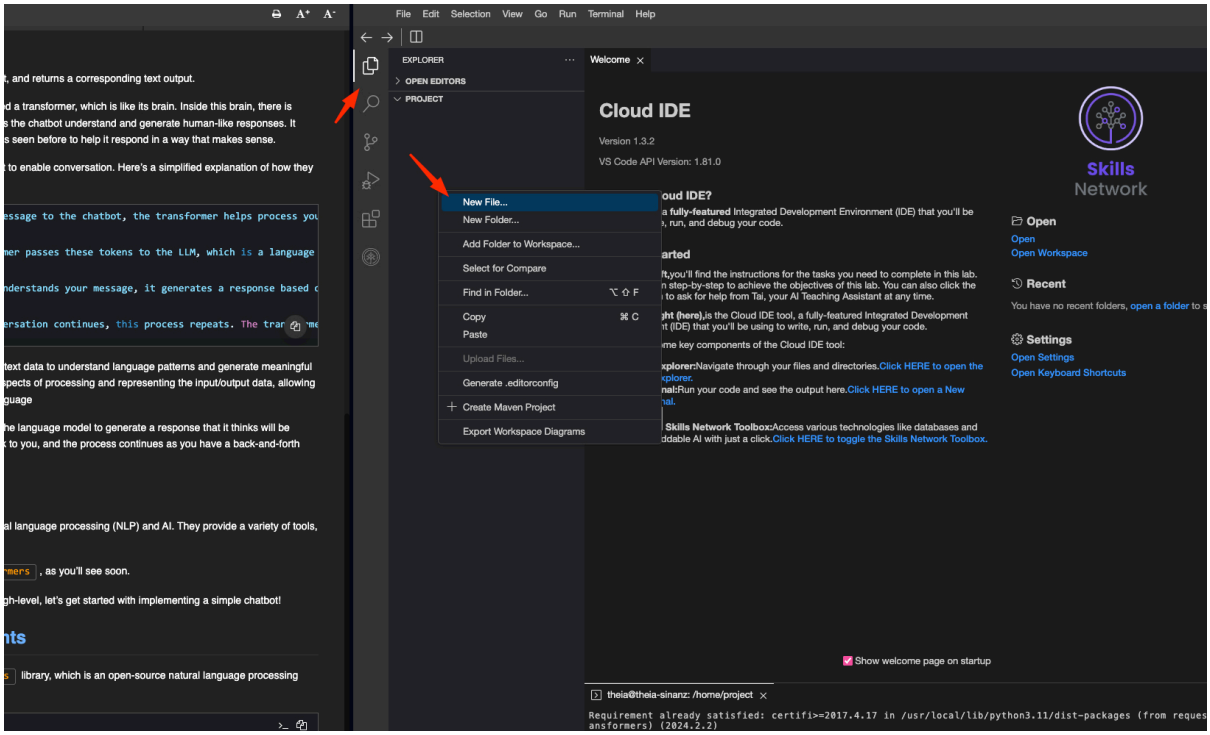
Run

Wait a few minutes to install the packages.



To create a new Python file, Click on `File Explorer`, then right-click in the explorer area and select `New File`. Name this new file `chatbot.py`.

Open chatbot.py in IDE



Part 1: Building a Simple Chatbot Using Transformer Models

Step 2: Import our required tools from the transformers library

For this example, you will be using `AutoTokenizer` and `AutoModelForSeq2SeqLM` from the `transformers` library.

python

```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
```

AutoTokenizer : converts text into tokens the model understands

AutoModelForSeq2SeqLM : loads a sequence-to-sequence generation model for dialogue

Add the step into the `chatbot.py` Python file.

Step 3: Choosing a model

Choosing the right model for your purposes is an important part of building chatbots! You can read on the different types of models available on the Hugging Face website: https://huggingface.co/models.

Models differ in architecture (encoder, decoder, or encoder-decoder), training methods (pretraining, fine-tuning, instruction tuning), and capabilities. Let's look at some examples to see how different models fit better in various contexts.

- **Text generation**: Text generation and chatbots have evolved over time with different types of models used depending on complexity and use case. Earlier systems used transformer-based encoder–decoder (Seq2Seq) models like facebook/blenderbot-400M-distill, as well as models such as T5 and BART.

These are lightweight, open-source, and can run on CPUs, making them suitable for simple chatbots and learning environments.

Modern chatbots, however, are built using large language models (LLMs) that use decoder-only transformer architectures and are trained on very large datasets. Examples include GPT-style models, LLaMA, and Mistral systems. These models are much more powerful in reasoning and conversation but require more computing resources or API access. Both approaches use transformers, but they differ in scale, training methods, and capability.

Example: You want to build a chatbot that generates creative and coherent responses to user input.

- **Sentiment analysis**: For sentiment analysis tasks, models like BERT or RoBERTa are popular choices. They are trained to understand the sentiment and emotional tone of text.

  Example: You want to analyze customer feedback and determine whether it is positive or negative.

- **Named entity recognition**: Models such as BERT or RoBERTa (fine-tuned for token classification) are commonly used for Named Entity Recognition (NER) tasks. They perform well in understanding and extracting entities like person names, locations, organizations, etc.

  Example: You want to build a system that extracts names of people and places from a given text.

- **Question answering**: Models like BERT (fine-tuned for QA) or modern instruction-tuned LLMs (e.g., GPT-4–class, LLaMA, Mistral) can be effective for question-answering tasks. They can comprehend questions and provide accurate answers based on the given context.

  Example: You want to build a chatbot that can answer factual questions from a given set of documents.

- **Language translation**: For language translation tasks, consider models like MarianMT, T5, or newer multilingual and instruction-tuned models such as mT5, NLLB, or modern LLMs. They are designed specifically for translating text between different languages.

Example: You want to build a language translation tool that translates English text to French.

However, these examples are very limited and the fit of an LLM may depend on many factors such as data availability, performance requirements, resource constraints, and domain-specific considerations. It's important to explore different LLMs thoroughly and experiment with them to find the best match for your specific application.

Other important purposes that should be taken into consideration when choosing an LLM include (but are not limited to):

- Licensing: Ensure you are allowed to use your chosen model the way you intend
- Model size: Larger models may be more accurate, but might also come at the cost of greater resource requirements
- Training data: Ensure that the model's training data aligns with the domain or context you intend to use the LLM for
- Performance and accuracy: Consider factors like accuracy, runtime, or any other metrics that are important for your specific use case

To explore all the different options, check out the available models on the Hugging Face website.

For this example, you'll be using `facebook/blenderbot-400M-distill`. This model is selected because:

It is open-source It is optimized for dialogue It is lightweight and runs efficiently on the CPU

plaintext

```
model_name = "facebook/blenderbot-400M-distill"
```

Add this step to your `chatbot.py` Python file.

Step 4: Fetch the model and initialize a tokenizer

When running this code for the first time, the host machine will download the model from Hugging Face API. However, after running the code once, the script will not re-download the model and will instead reference the local installation.

You'll be looking at two terms here: `model` and `tokenizer`.

In this script, you initiate variables using two handy classes from the `transformers` library:

- `model` is an instance of the class `AutoModelForSeq2SeqLM`, which allows you to interact with your chosen language model.
- `tokenizer` is an instance of the class `AutoTokenizer`, which optimizes your input and passes it to the language model efficiently. It does so by converting your text input to "tokens", which is how the model interprets the text.

markdown

```
# Load model (download on first run and reference local installation for s
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)
tokenizer = AutoTokenizer.from_pretrained(model_name)
```

Conversation details

Step 5: Chat

Now that you're all set up, let's start chatting!

There are several things you'll do to have an effective conversation with your chatbot.

Before interacting with your model, you need to initialize an object where you can store your conversation history.

1. Initialize an object to store the conversation history

Afterward, you'll do the following for each interaction with the model:

2. Encode conversation history as a string
3. Fetch prompt from user
4. Tokenize (optimize) prompt
5. Generate output from the model using prompt and history
6. Decode output
7. Update conversation history

Step 5.1: Keeping track of conversation history

The conversation history is important when interacting with a chatbot because the chatbot will also reference the previous conversations when generating output.

For your simple implementation in Python, you may use a list. Per the Hugging Face implementation, you will use this list to store the conversation history as follows:

> *conversation_history*
>
> *conversation_history [User: input_1, Bot: output_1, User: input_2, Bot: output_2, ...]*

plaintext

```
conversation_history = []
```

Let's print a simple message which will help you to quit the chatbot once the whole code is ready:

python

```python
print("Chatbot ready! (type 'exit' to quit)\n")
```

Add this step to your Python code in chatbot.py

Step 5.2: Encoding the conversation history

During each interaction, you will pass your conversation history to the model along with your input so that it may also reference the previous conversation when generating the next answer.

The `transformers` library function you are using expects to receive the conversation history as a string, with each element separated by the newline character `'\n'`. Thus, you create such a string.

You'll use the `join()` method in Python to do exactly that. (Initially, your history_string will be an empty string, which is okay, and will grow as the conversation goes on).

python

```python
history_string = "\n".join(conversation_history)
```

Add this to `chatbot.py`

Step 5.3: Fetch prompt from user

Before you start building a simple terminal chatbot, let's look at an example of the input:

```python
input_text = input("> ")
```

Add this to `chatbot.py`

Step 5.4: Tokenization of user prompt and chat history

Tokens in NLP are individual units or elements that text or sentences are divided into. Tokenization or vectorization is the process of converting tokens into numerical representations. Tokenization converts text into a numerical format that the model can understand

In this implementation, we pass both history and new input together as a single input. This line creates a single prompt by combining the conversation history with the latest user input. The format `User: ...` and `Bot:` clearly separates roles and signals the model to generate the bot's next response. The trailing `Bot:` helps the model continue text in assistant style.

```plaintext
prompt = history_string + f"\nUser: {input_text}\nBot:"

inputs = tokenizer(
    prompt,
    return_tensors="pt",
    truncation=True,
    max_length=512
)
```

- **tokenizer(...):** Converts raw text into numerical tokens the model can understand.
- **history_string:** Previous conversation history used to provide context.
- **input_text:** Current user message.
- **return_tensors="pt":** Returns PyTorch tensors (required by the model).
- **truncation=True:** Truncates input if it exceeds model limits.
- **max_length=512:** Maximum number of tokens allowed as input.

plaintext

```
python3 chatbot.py
```

In doing so, you've now created a tokenized tensor dictionary-like object which contains special keywords that allow the model to reference its contents properly.

Step 5.5: Generate output from the model

Now that you have your inputs ready, both past and present inputs, you can pass them to the model and generate a response. According to the documentation, you can use the `generate()` function and pass the inputs as keyword arguments (kwargs).

python

```
outputs = model.generate(
    **inputs,
    max_new_tokens=60,
    no_repeat_ngram_size=3,
    repetition_penalty=1.3,
    do_sample=True,
    temperature=0.6,
    top_p=0.85
)
## Remove this print statement after testing
print(outputs)
```

- **inputs**: Sends the user message and chat history to the model. This helps the chatbot understand the full conversation before replying.
- **max_new_tokens**: Sets the maximum length of the reply. It stops the model from writing too much text.
- **no_repeat_ngram_size**: Stops the model from repeating the same 3-word phrases again and again.
- **repetition_penalty**: Reduces repeated words in the response so the output sounds more natural.
- **do_sample=True**: Makes the chatbot responses more random and less fixed, so replies feel more natural.
- **temperature**: Controls how creative the response is. Lower = safer answers

- **top_p**: Keeps only the most likely word choices when generating text, which helps the response stay clear and meaningful.

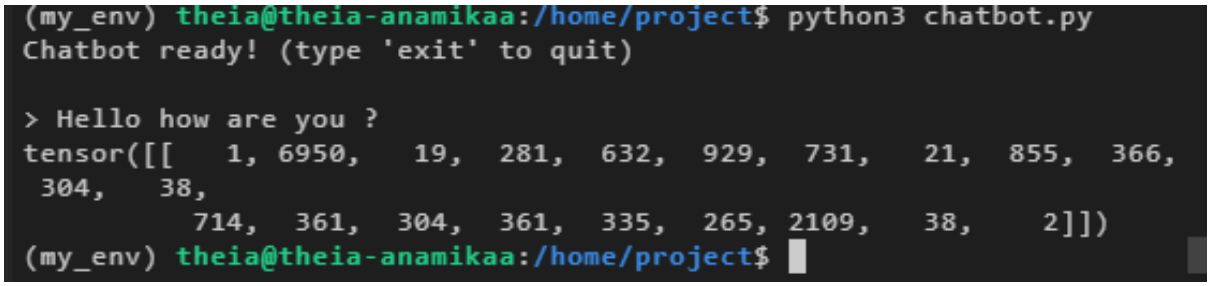
Add this to `chatbot.py` and run it:

Start the conversation by asking `Hello how are you?`

```bash
python3 chatbot.py
```

Run

The output:



Great - now you have your outputs! However, the output contains token IDs (tensor values), not readable text.

Therefore, you just need to decode the first index of `outputs` to see the response in plaintext.

Step 5.6: Decode output

You may decode the output using `tokenizer.decode()`. This is known as "detokenization" or "reconstruction". It is the process of combining or merging individual tokens back into their original form, to reconstruct the original text or sentence.

```python
response = tokenizer.decode(outputs[0], skip_special_tokens=True).strip()
print(response)
```

- **tokenizer.decode(outputs[0])**: Converts the model's output from numbers (tokens) back into readable text. The model first generates numbers, and this

- **outputs[0]**: Takes the first generated response from the model (since the model can generate multiple outputs internally).
- **skip_special_tokens=True**: Removes special tokens like padding or system symbols so they don't appear in the final output.
- **.strip()**: Removes extra spaces at the beginning and end of the text for a clean output.
- **print(response)**: Displays the final chatbot reply in the terminal so the user can see it.

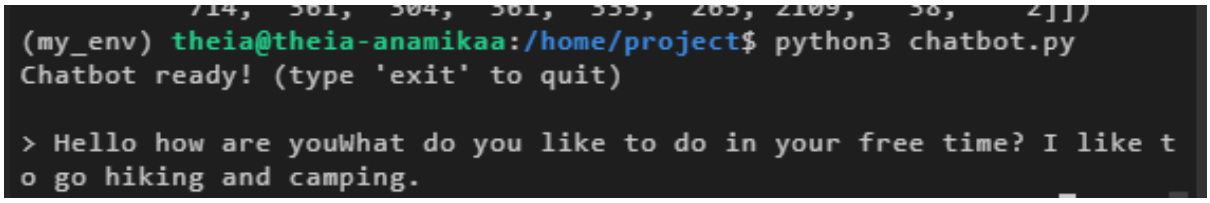
Add this to `chatbot.py` and run it:

Start the conversation by asking `Hello how are you?`

```bash
python3 chatbot.py
```

Run

The output:



Alright! You've successfully had an interaction with your chatbot! You've given it a prompt and received its response.

Now, all that's left to do is to update your conversation history, so that you may pass it with the next iteration.

Step 5.7: Update conversation history

All you need to do here is add both the input and response to `conversation_history` in plaintext.

```python
conversation_history.append(f"User: {input_text}")
conversation_history.append(f"Bot: {response}")
print(conversation_history)
```

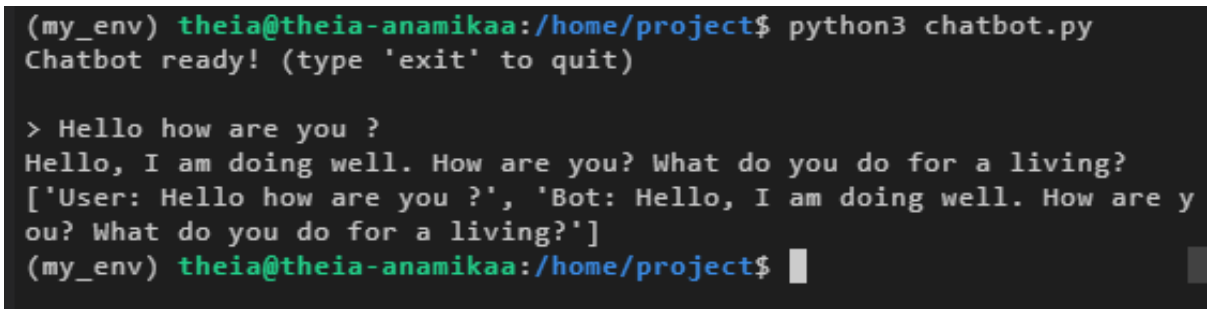
Add this to `chatbot.py` and run it:

Start the conversation by asking `Hello how are you?`

```bash
python3 chatbot.py
```

Run

The output



Step 6: Repeat

You have gone through all the steps of interacting with your chatbot. Now, you can put everything in a loop and run a whole conversation!

We have further enhanced it by adding

```python
    # keep only last few exchanges (prevents confusion)
    conversation_history = conversation_history[-6:]
```

because the model can only handle a limited amount of text at once. In a long conversation, older messages are not always useful for generating the next response and can even confuse the model or dilute the context. By keeping only the

- The chatbot focuses on the most recent and relevant part of the conversation
- It avoids getting overwhelmed by long chat history
- It reduces repetition and improves response quality
- It keeps the input within the model's token limit

Now,we neeed to add everything in a loop so that conversation keep flowing.

python

```
while True:
    # keep only last few exchanges (prevents confusion)
    conversation_history = conversation_history[-6:]

    history_string = "\n".join(conversation_history)

    input_text = input("> ")
## This will help you exit by typing exit in the prompt
    if input_text.lower() == "exit":
        break

    prompt = history_string + f"\nUser: {input_text}\nBot:"

    inputs = tokenizer(
    prompt,
    return_tensors="pt",
    truncation=True,
    max_length=512
)

    outputs = model.generate(
        **inputs,
        max_new_tokens=60,
        no_repeat_ngram_size=3,
        repetition_penalty=1.3,
        do_sample=True,
        temperature=0.6,
        top_p=0.85
    )

    response = tokenizer.decode(outputs[0], skip_special_tokens=True).stri
    print("Bot:", response)

    conversation_history.append(f"User: {input_text}")
    conversation_history.append(f"Bot: {response}")
```

Add this to `chatbot.py` and run it:

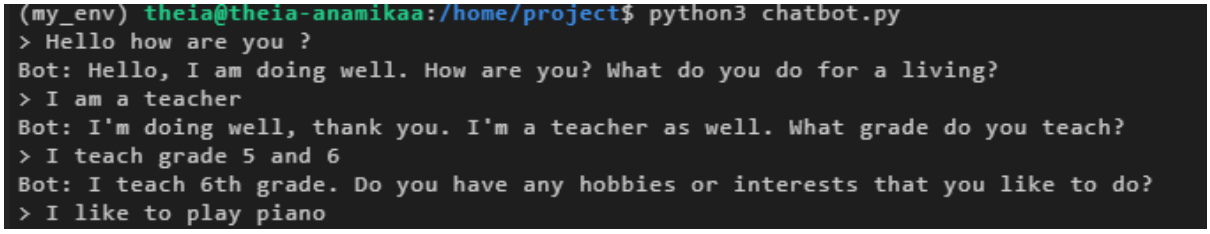
Start the conversation by asking `Hello how are you?`

```bash
python3 chatbot.py
```

Run

> ***Note:** The model used in this project is a basic, lightweight version, not intended for handling complex queries and it may produce generic or inconsistent responses during extended conversations. For more advanced and robust LLMs, you can explore a wide range of options at huggingface.com.*

The output:



This will be the final solution:

▶ Click to expand

Voila! You have built a simple, functional chatbot that you can interact with through your terminal!

Press `cntrl + c` to exit the conversation.Or just type `exit` in the prompt.

Part 2: Building a Modern LLM Chatbot with Chat Templates

In Part 1, you build a simple chatbot using a Seq2Seq model. You manually create the input text, convert it into tokens, and store the chat history yourself. This helps you understand the basic working of a chatbot and how transformers generate replies.

In Part 2, you use a more modern AI model (a causal LLM). Here, you don't manually build prompts. Instead, you use structured messages like "user" and "assistant", and the model handles the conversation format itself.

Step 1: Import required libraries

At first, start by creating a new file named `chatbot_llm.py`

Import the required libraries

python

```python
from transformers import AutoTokenizer, AutoModelForCausalLM
import torch
import warnings

warnings.filterwarnings("ignore")
```

Add this step to `chatbot_llm.py`

AutoTokenizer for tokenization

AutoModelForCausalLM for loading a causal language model

torch for model inference

warnings.filterwarnings("ignore") suppresses unnecessary Hugging Face warning messages to keep the output cleaner.

Step 2: Choose a modern LLM

For this example, you will use:

```python
model_name = "HuggingFaceTB/SmolLM2-360M-Instruct"
```

Add this step to `chatbot_llm.py`

This is a lightweight instruction-tuned causal LLM designed for conversational tasks.

Step 3: Load model and tokenizer

```python
print("Loading model...")

tokenizer = AutoTokenizer.from_pretrained(model_name)

tokenizer.pad_token = tokenizer.unk_token

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    device_map="cpu",
    torch_dtype=torch.float32
)
```

Add this step to `chatbot_llm.py`

- **pad_token**: In transformer models, inputs in a batch must often be the same length. Shorter sequences are padded with a special token called the padding token (pad_token). This tells the model which parts of the input are real words and which are filler.
- **device_map**: Controls where the model runs (e.g., CPU or GPU) and ensures it is correctly loaded on the available device.
- **torch_dtype**: Sets the numerical precision of computations (e.g., float32 or float16) to balance speed, memory usage, and accuracy.

Step 4: Initialize conversation messages

In modern chat-based LLMs, we use a structured conversation format made of messages. Each message has a specific role that tells the model who is speaking and how to behave.

```python
messages = [
    {
        "role": "system",
        "content": "You are a helpful AI assistant. Give short and concise
    }
]
```

Add this step to `chatbot_llm.py`

messages: This is the full conversation history between the user and the AI. Each message has two parts: **role** who is speaking and **content** what they are saying

There are three types of **roles** in a chat-based AI system. The **system** role defines the rules and behavior of the AI, such as how it should respond. The **user** role represents the questions or inputs given by the person using the chatbot. The **assistant** role contains the AI's responses generated based on both the system instructions and user input, forming the conversation flow.

Step 5: Start chatbot loop

Add these steps to `chatbot_llm.py`

```python
print("Chatbot started. Type 'exit' to quit.\n")
while True:
  user_input = input("> ")

  if user_input.lower() == "exit":
      break
```

Step 5.1: Update conversation history

Store the user message in the conversation history.

```python
messages.append({"role": "user", "content": user_input})
```

To avoid very long conversations, keep only recent exchanges:

```python
messages = [messages[0]] + messages[-10:]
```

This keeps:

- the system message
- the latest conversation history

Step 5.2 : Apply chat template

Modern Hugging Face chat models use chat templates to format conversations automatically.

```python
tokenized = tokenizer.apply_chat_template(
    messages,
    tokenize=True,
    add_generation_prompt=True,
    return_tensors="pt",
    return_dict=True,
    max_length=512
)
```

- **apply_chat_template()**: This function converts the structured message format (system, user, assistant) into a single properly formatted prompt that the model can understand. It ensures the conversation follows the model's expected template, including special tokens and structure.
- **tokenize**: converts text into tokens

- **add_generation_prompts**: signals the model to generate a reply This flag tells the model that the input ends here and it should now start generating a response. It ensures the model knows where the assistant's reply should begin in the conversation format.
- **return_tensors**: returns PyTorch tensors

Step 5.3: Generate response

This code is used to generate a response from a language model based on the given input text.It takes your tokenized input and makes the model predict and generate a reply token-by-token, while controlling the style, length, and randomness of the output

python

```python
with torch.inference_mode():
    outputs = model.generate(
        tokenized["input_ids"],
        attention_mask=tokenized["attention_mask"],
        max_new_tokens=60,
        temperature=0.5,
        top_p=0.8,
        do_sample=True,
        repetition_penalty=1.3,
        no_repeat_ngram_size=3,
        pad_token_id=tokenizer.pad_token_id
    )
```

- **with torch.inference_mode()**:Runs the code in inference mode (no training). It makes generation faster and memory-efficient.
- **model.generate()**: This is the function that actually makes the model produce a response based on the input tokens.
- **tokenized["input_ids"]**: These are the numerical tokens representing your input text. The model uses them as the starting point for generating output.
- **attention_mask=tokenized["attention_mask"]** : This tells the model which tokens are real words and which are padding. It ensures the model ignores padding tokens during generation.
- **pad_token_id=tokenizer.pad_token_id** :Specifies the padding token ID so the

Step 5.4: Decode and display response

After the model generates output, it is still in token form (numbers). This step converts it back into readable text and shows it to the user.

```python
response = tokenizer.decode(
    outputs[0][tokenized["input_ids"].shape[-1]:],
    skip_special_tokens=True
)
print(f"Bot: {response}\n")
```

outputs[0][tokenized["input_ids"].shape[-1]:] This part extracts only the newly generated response from the model.

- **outputs[0]** :full sequence (input + generated text)
- **tokenized["input_ids"].shape[-1]** : length of original input
- **[...] **: slices out only the generated part (removes the input)

So the model doesn't repeat the user's question in the output.

Step 5.5 : Save assistant response

```python
messages.append({"role": "assistant", "content": response})
```

In case you have missed,add the while loop code of step 5 to `chatbot_llm.py`

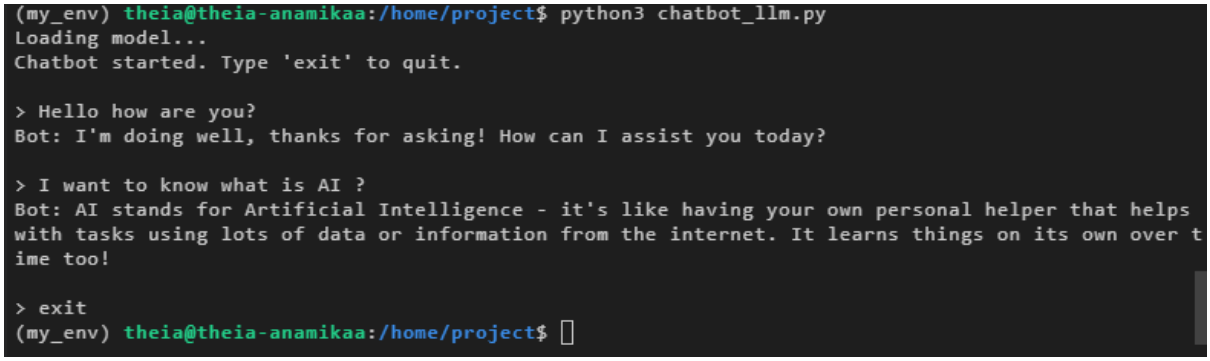
▶ Click to expand

Run the lab. Start the conversation by asking `Hello, how are you?`

```bash
python3 chatbot_llm.py
```

Run

The output:



You have now built a modern chatbot using a causal LLM and Hugging Face chat templates. This workflow is similar to how many modern AI assistants and conversational systems are implemented today.

You can experiment with different settings to see how the bot's behavior changes. This is similar to how real AI systems are tuned to feel more friendly, creative, or strict depending on the use case. For example, you can make the bot more friendly by changing the system prompt:

plaintext

```
messages = [{
    "role": "system",
    "content": "You are a very friendly and cheerful assistant. Always res
}]
```

Then try changing generation parameters to see the difference in responses:

plaintext

```
temperature=0.9
top_p=0.95
```

This is the final solution for your reference:

▶ Click to expand

Conclusion

Congratulations, you have successfully completed this lab! This lab showcases the creation of a basic chatbot using Python and Hugging Face's open-source language models. The project guides learners through the key concepts of chatbots, including understanding and using transformers and tokenization. It provides a step-by-step approach to setting up the environment, choosing a model, and coding a chatbot capable of holding a conversation. This hands-on project, crafted by Dr. Sina Nazeri, is a great resource for beginners interested in exploring AI and chatbot development, demonstrating how to build a functional chatbot in an accessible and engaging way.

Authors

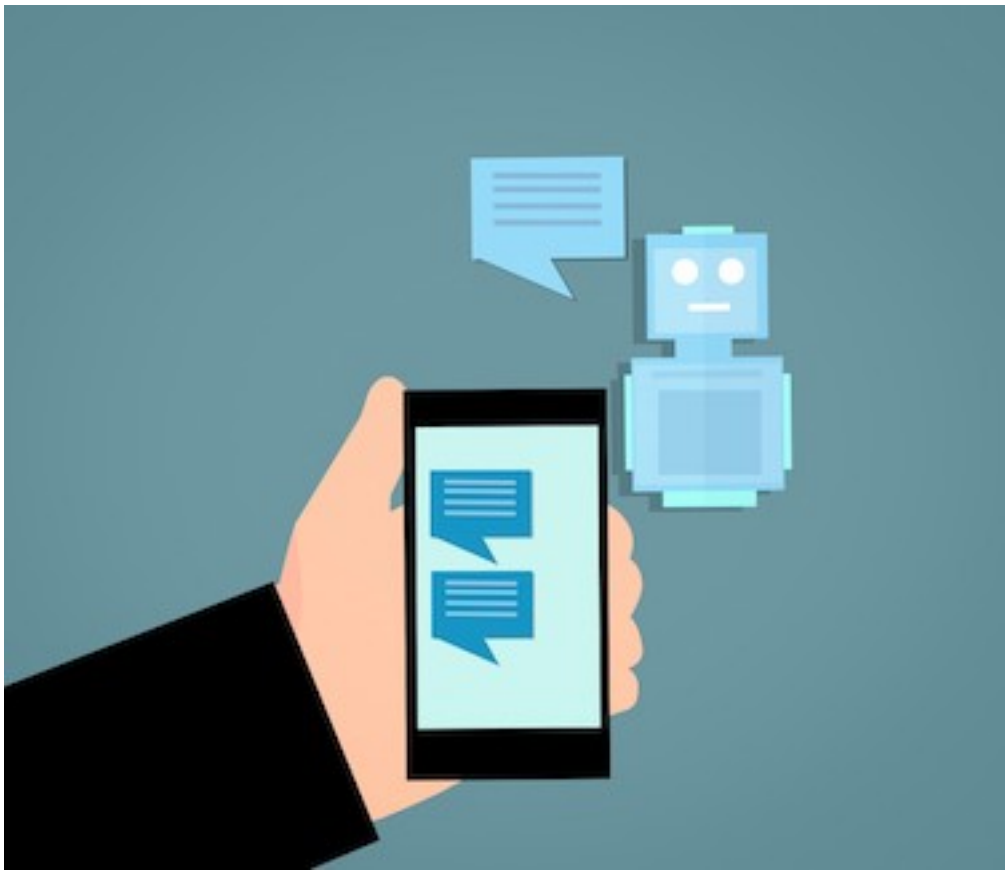
Vandana Pandey

Sina Nazeri (Ph.D.)

| As a data scientist in IBM, I have always been passionate about sharing my knowledge and helping others learn about the field. I believe that everyone should have the opportunity to learn about data science, regardless of their background or experience level. This belief has inspired me to become a learning content provider, creating and sharing educational materials that are accessible and engaging for everyone. |

Create Simple Chatbot with Open Source LLMs using Python and Hugging Face

In this lab, you will create a very simple but functional chatbot!



Learning outcomes:

At the end of this lab, you will be able to:

- Explain what an LLM is
- Select an LLM for your application
- Describe how a transformer essentially works
- Feed input into a transformer (tokenization)
- Program your own simple chatbot in Python

Introduction: Under the hood of a chatbot

Intro: How does a chatbot work?

A chatbot is a computer program that takes a text input, and returns a corresponding text output.

Chatbots use a special kind of computer program called a transformer, which is like its brain. Inside this system is a language model (LLM), which is the core component that generates resposes.This helps the chatbot understand and generate human-like responses. It deciphers many examples of human conversations it has seen prior to responding in a sensible manner.

Transformers and LLMs work together within a chatbot to enable conversation. Here's a simplified explanation of how they interact:

- **Input processing:** When you send a message to the chatbot, the transformer helps process your input. It breaks down your message into smaller parts and represents them in a way that the chatbot can understand. Each part is called a token.

- **Understanding context:** The transformer passes these tokens to the LLM, which is a language model trained on lots of text data. The LLM has learned patterns and meanings from this data, so it tries to understand the context of your message based on what it has learned.

- **Generating response:** Once the LLM understands your message, it generates a response based on its understanding. The transformer then takes this response and converts it into a format that can be easily sent back to you.

- **Iterative conversation:** As the conversation continues, this process repeats. The transformer and LLM work together to process each new input message, understand the context, and generate a relevant response.

The key is that the LLM learns from a large amount of text data to understand language patterns and generate meaningful responses. The transformer helps with the technical aspects of processing and representing the input/output data, allowing the LLM to focus on understanding and generating language.

Once the chatbot understands your message, it uses the language model to generate a response that it thinks will be helpful or interesting to you. The response is sent back to you, and the process continues as you have a back-and-forth conversation with the chatbot.

Intro: Hugging Face

Hugging Face is an organization that focuses on natural language processing (NLP) and AI. They provide a variety of tools, resources, and services to support NLP tasks.

You'll be making use of their Python library `transformers` in this project.

Alright! Now that you know how a chatbot works at a high level, let's get started with implementing a simple chatbot!

Step 1: Installing requirements

Follow these steps to create a Python virtual environment and install the necessary libraries. Open a new terminal first. Set up your virtual environment:

bash

```
pip3 install virtualenv
virtualenv my_env # create a virtual environment my_env
source my_env/bin/activate # activate my_env
```

Run

For this example, we use the transformers library, an open-source NLP toolkit, along with PyTorch (torch) for deep learning, while accelerate helps run AI models efficiently on CPU/GPU and numpy supports fast numerical and array computations in Python.

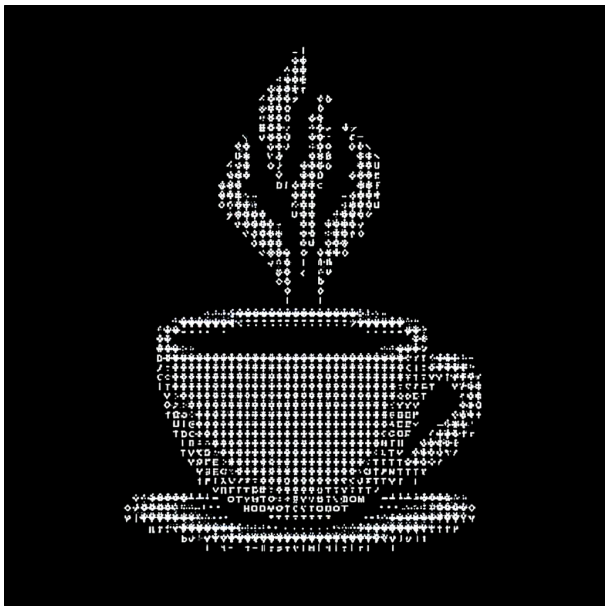
We pin library versions to ensure the code runs consistently without breaking due to future updates or changes in dependencies.

bash

```
pip install transformers==4.41.2 torch==2.2.2 accelerate==0.30.1 numpy==1.
```

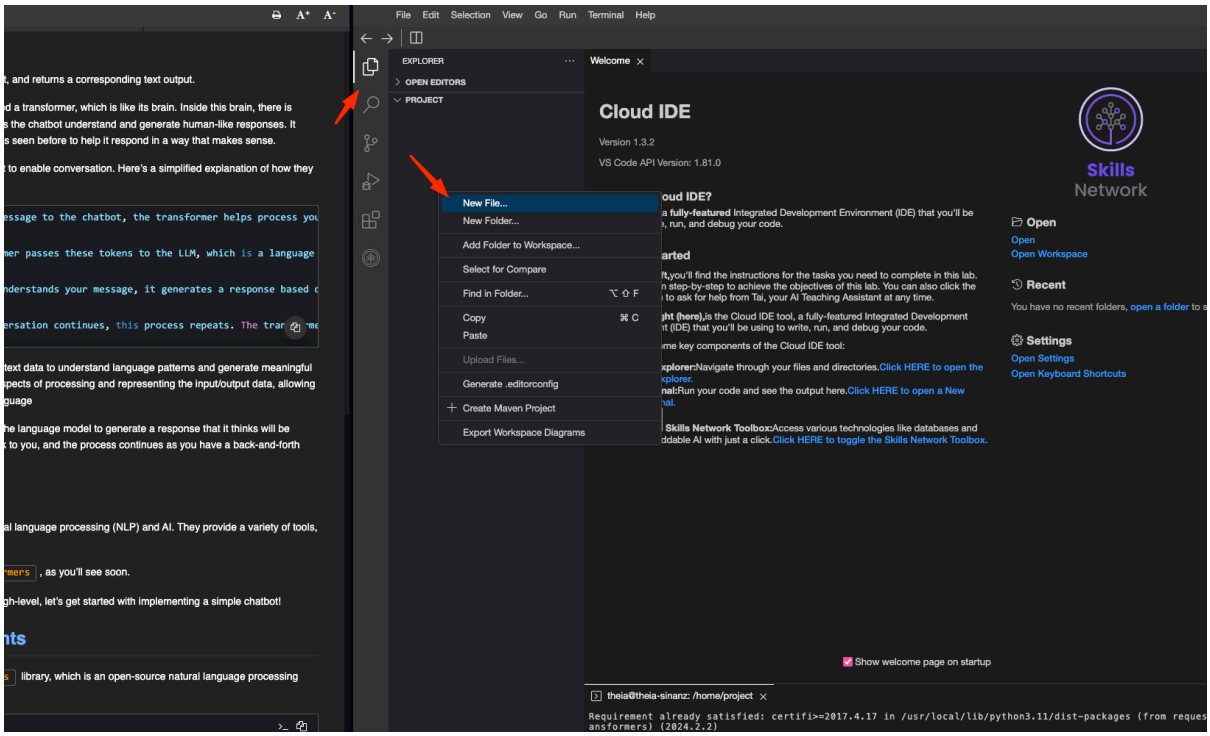
Run

Wait a few minutes to install the packages.



To create a new Python file, Click on `File Explorer`, then right-click in the explorer area and select `New File`. Name this new file `chatbot.py`.

Open chatbot.py in IDE



Part 1: Building a Simple … →